

Report

Name: Ansh Pachauri

Student ID: Pachauri.2

Class: CSE 3521

HW 4, Programming part

Output from Linear_Classifiers.py --data simple --algorithm logistic:

```
● (base) anshpachauri@Anshs-MacBook-Pro HW_4_Programming % python3 Linear_Classifiers.py --data simple --algorithm logistic
number of training data instances: (3, 8)
number of test data instances: (3, 2)
number of training data labels: (8, 1)
number of test data labels: (2, 1)
/Users/anshpachauri/Desktop/College/Spring_2025/CSE3521_Intro_to_AI/HW_4_Programming/Linear_Classifiers.py:153: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
    return float(sum(correct)) / len(correct)
Accuracy: training set: 1.0
Accuracy: test set: 1.0
```

Output from Linear_Classifiers.py --data starplus --algorithm logistic:

```
● (base) anshpachauri@Anshs-MacBook-Pro HW_4_Programming % python3 Linear_Classifiers.py --data starplus --algorithm logistic
number of training data instances: (258391, 40)
number of test data instances: (258391, 14)
number of training data labels: (40, 1)
number of test data labels: (14, 1)
/Users/anshpachauri/Desktop/College/Spring_2025/CSE3521_Intro_to_AI/HW_4_Programming/Linear_Classifiers.py:153: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
    return float(sum(correct)) / len(correct)
Accuracy: training set: 1.0
Accuracy: test set: 0.7142857142857143
```

Output from Linear_Classifiers.py --data simple --algorithm perceptron:

```
● (base) anshpachauri@Anshs-MacBook-Pro HW_4_Programming % python3 Linear_Classifiers.py --data simple --algorithm perceptron
number of training data instances: (3, 8)
number of test data instances: (3, 2)
number of training data labels: (8, 1)
number of test data labels: (2, 1)
/Users/anshpachauri/Desktop/College/Spring_2025/CSE3521_Intro_to_AI/HW_4_Programming/Linear_Classifiers.py:153: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
    return float(sum(correct)) / len(correct)
Accuracy: training set: 1.0
Accuracy: test set: 1.0
```

Output from Linear_Classifiers.py --data starplus --algorithm perceptron:

```
● (base) anshpachauri@Anshs-MacBook-Pro HW_4_Programming % python3 Linear_Classifiers.py --data starplus --algorithm perceptron
number of training data instances: (258391, 40)
number of test data instances: (258391, 14)
number of training data labels: (40, 1)
number of test data labels: (14, 1)
/Users/anshpachauri/Desktop/College/Spring_2025/CSE3521_Intro_to_AI/HW_4_Programming/Linear_Classifiers.py:153: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
    return float(sum(correct)) / len(correct)
Accuracy: training set: 1.0
Accuracy: test set: 0.7857142857142857
```

In this assignment, I implemented both logistic regression and perceptron classifiers using gradient-based and mistake-driven learning respectively. For logistic regression, I computed the average gradient of the logistic loss and updated weights using batch gradient descent, as shown in the slide titled “this is the main thing you need to compute.” The sigmoid function was applied to the linear combination of features and weights, and gradients were calculated efficiently using matrix operations.

For the perceptron, I followed the slide's description of checking the sign of the dot product between the weight vector and each input, updating the weights only if a misclassification occurred. Both implementations were restricted to under 10 lines within the provided TODO blocks, and I maintained the exact structure of the skeleton code. The models achieved

the expected accuracies on both simple and starplus datasets, demonstrating that the implementations correctly followed the mathematical logic and update rules provided in class.